

Resumen Organización de Datos P1

Unidad 1 - Organización de archivos

Se utilizan técnicas para organizar datos con el objetivo de optimizar la eficiencia de almacenamiento, recuperación y resguardo de datos.

- **Estructura de archivos:** Cómo están constituidos físicamente los archivos.
- **Organización de archivos:** Administración de los archivos, en función de las relaciones de los registros.

Sistema operativo: Responsable del manejo de recursos de una computadora. Los recursos pueden ser físicos (hardware) o lógicos (software). Los programas y usuarios tienen una visión lógica de la información, y los dispositivos de almacenamiento una visión física. El sistema operativo es el mediador entre ambos.

Sistema de Archivo se denomina al conjunto de programas del sistema operativo que se encargan de proveer la visión lógica de la información almacenada a usuarios y programas.

❖ Funciones del SO:

- Identificar y ubicar archivos.
- Brindar seguridad.
- Asignar espacio en dispositivos de almacenamiento.
- Coordinar transferencias.
- Administrar la comunicación con CPU y dispositivos de almacenamiento.

Dato: Representación de una cosa real o evento ocurrido en una unidad lógica de manipulación llamada registro, que se representan en unidades llamadas campos. Los atributos de los registros son datos.

Archivo de Datos: Unidad lógica de almacenamiento permanente de registros, administrada por un SO.

Los registros, dentro del archivo, se pueden organizar en otras unidades lógicas llamadas bloques o páginas.

❖ Clasificación de archivos según su **función**:

- **Datos maestros:** Datos de un sistema de información que representan entidades de existencia real o ideal.
- **Datos transaccionales:** Registros de hechos o eventos relacionados con datos maestros.
- **Reporte:** Información editada para presentar a un usuario (pdf,html o texto)
- **Trabajo:** Resultados parciales, intermedios, o datos de intercambio entre programas.
- **Control de datos:** Almacén de metadatos (definiciones de datos), administra espacios libres, accede al contenido de otros archivos, etc.
- **Intercambio de datos:** Se representan datos en formatos estándar para que puedan ser procesados y manipulados (ej:archivos de texto con tablas)
- **Recursos de programa:** Unidades grandes de información (imágenes, audio, video)

- **Productos de programas:** Archivos asociados a otros programas o aplicaciones (word, excel, etc)
- **Empaquetado de archivos:** Se utiliza para agrupar archivos y directorios mediante compresión (zip,rar,etc)

❖ Clasificación de archivos según su **tipo de Datos:**

- **Texto (ASCII).**
- **Binarios.**
- **Tipificados y sin tipo.**

❖ Clasificación de archivos por **tipo de registros:**

- **Registro de longitud fija:** Elementos del mismo tamaño. Se introducen de manera consecutiva.
-Ventaja: Facilita la localización y extracción de la información del campo.
-Desventaja: Tamaño del campo produce desperdicio de espacio (puede llegar a no utilizarse todo el registro).
- **Registro de longitud variable:** Almacenan elementos de varios tipos en un archivo, permitiendo campos de longitudes variables y campos repetidos.
La longitud de cada registro tiene que estar definida para poder leer y escribir de forma efectiva.
-Ventaja: Aprovechamiento del espacio.
-Desventaja: Dificultad para la localización y extracción de la información.

❖ Clasificación de archivos por **tipo de acceso:**

- **Secuencial:** Registro a continuación de otro.
Para agregar registros hay dos opciones: Crear un nuevo archivo o agregar al final.
Para eliminar registros se pueden marcar o se debe crear un nuevo archivo.
Ventaja: Tamaño y tiempo de acceso mínimo.
- **Directo:** Acceso de manera rápida y simple a los registros de un archivo.
La secuencia u ordenamiento lógico no necesariamente tiene una relación con la secuencia física.
Para acceder a los registros es a través de la **clave** de dicho archivo. **La clave o llave principal** es el campo del archivo que permite identificar un registro de otro.
La memoria principal y los discos magnéticos son la mejor opción. Las cintas son lo peor.
La complejidad de este tipo se basa en programar la relación entre el contenido de un registro y su posición física en el archivo.
-Suelen existir huecos libres entre registros.
-Cada clave es única y no puede tener duplicado.
-Es preferente que las claves tengan un orden ascendente.
-Los valores de las claves están en un rango acotado.
Se genera un problema si las claves de un registro no representan una dirección.
- **Indexado:** Conjunto de registros que pueden accederse mediante una clave.
Presenta dos áreas.
 - **Área principal:** Aquí se almacenan los registros con los datos, al momento de crear el archivo.

- **Área de índices:** Aquí hay registros con dos campos. Uno para la clave de los registros, y otro con el puntero al registro en el área principal.
Para eliminar basta con poner un puntero a null en el área índices.
Para insertar puede hacerse anexando o aprovechando una posición eliminada.
- **Secuenciales indexados:** El índice se puede organizar de diversas formas. Las más típicas son: secuencial, multinivel y árbol.
Se puede tener tantos índices como se quiera variando la clase que se emplee.
- **Relativos.**
- **Stream (flujo de datos).**

Visión lógica de los archivos

Los registros que los componen pueden tener los siguientes atributos::

-Nombre, Tamaño, Fecha/Hora, Dueño, Grupo, etc.

Las operaciones que se pueden realizar en estos registros son:

-Crear, eliminar, abrir, cerrar, leer, obtener atributos, definir atributos, etc.

Diseño de datos

Se dividen en dos etapas o fases:

1. Diseño conceptual o de alto nivel: Definir atributos, sin especificar tipos o dominios.
2. Diseño lógico o de bajo nivel: Definir bloques o unidades de organización dentro del archivo.

Ambas fases son independientes del lenguaje de programación.

1. Diseño conceptual

Atributos: Especificación que define una propiedad de un objeto, elemento o archivo.
También se puede referir al valor específico para una instancia determinada.

❖ Tipos de atributos:

- **Simple:** Una propiedad (nombre, apellido, altura, etc).
- **Compuestos:** Una propiedad compuesta (fecha compuesta por día, mes y año)
- **Opcional:** Puede o no estar al momento de registrarse.
- **Polivalentes:** Depende de una lista de valores del mismo tipo.
- **Externos:** Relacionan un dato con otro o otros del mismo tipo o conjunto.

Identificador: Cada cosa o evento debe poder **distinguirse** de los demás.
Debe haber al menos un atributo que identifique unívocamente a los datos.

LOS CALIFICADORES

DE IDENTIFICACIÓN		DE CARDINALIDAD	
i	Identificador	?	Opcional
ie	Identificador externo.	*	Ninguno o varios (0 a N)
d	Definido por extensión en otro archivo	+	Uno o varios (1 a N)

2. **Diseño lógica:** Se aplica para cada atributo. Este diseño implica la definición de bloques o unidades de organización, especificando tipos o dominios para atributos, y determina la convención de almacenamiento y recuperación.

Al definir el tipo para cada atributo permite saber de antemano cuánto espacio va a ocupar cada registro en el archivo, generando la definición de bloques o unidades de organización.

Primitivas de Organización de Archivos

- **De creación:** Crear y cargar sin validar unicidad ni búsqueda de espacio libre.
- **De actualización de registros:** Insertar con validación de unicidad y búsqueda de espacio libre, modificación y supresión
- **De recuperación de registros:** Consulta o recuperación unitaria de registros.
- **De mantenimiento:** Reestructuración, depuración y respaldo.

Criterios para Elegir un tipo de Organización

- **Rápido acceso:** Costo en tiempo de acceso al dato.
- **Economía de almacenamiento:** Costo en tamaño de uso del espacio disponible.
- **Facilidad de uso:** Complejidad algorítmica.

La elección de la organización determina el rendimiento relativo del sistema.

Medidas para determinar **rendimiento relativo**:

- Almacenamiento requerido por un registro.
- Tiempo de búsqueda de un registro.
- Tiempo requerido para leer todo el archivo.
- Tiempo requerido para insertar un registro.
- Tiempo para modificar un registro.

Datos del Tintero (si, esto también entra)

Datos sobre Accesos:

Parámetros Físicos:

- nº cilindros
- pistas por cilindro (cabezas)
- sectores por pista
- bytes por sector
- tamaño bloque control
- velocidad rotación (rpm) _
- velocidad desplaz. Cabezas
- + tiempo arranque

Parámetros Lógicos:

- factor intercalado
- factor bloque (reg. / bloque)

Parámetros Temporales:

- tiempo retraso rotación T_{RR}
- tiempo transferencia T_{TR}
- tiempo desplazamiento T_{DS}

Tiempo Desplazamiento cabezas: $t_{dm} = n^{\circ} \text{ cilindros} \times v_{dc} + t_{arr}$

Datos para cálculos de espacio y tiempo en discos

- Capacidad de disco: $b * s * p * c$ siendo:
b: Cantidad de bytes por sector.

s: Cantidad de sectores por pista.

p: Cantidad de pistas por cara.

c: Cantidad de caras.

- Tiempo de búsqueda (Tpos): Tiempo que se demora en desplazar la cabeza hasta las pista (cilindro) que se requiere acceder.
- Tiempo de latencia (Tl): Tiempo que se tarda en ubicar al sector requerido.
- Tiempo de acceso (Ta): Tiempo que se tarda hasta que los datos requeridos estén justo debajo de la cabeza.

$$Ta = Tpos + Tl$$

- Tiempo de transferencia (Ts): Tiempo que demora el disco en transferir un sector a la memoria.

$$Ts = \text{Tiempo de 1 vuelta} / \text{Cantidad de sectores de una pista.}$$

- Tiempo de lectura (Tr): Tiempo que demora el disco en leer los datos.

$$Tr = Ta + Ts * Fb$$

**Fb: Cantidad de sectores leídos continuos.

- Tasa de Actividad = $\frac{\text{Nro. reg. accedidos}}{\text{Nro. reg. fichero}}$
- Volatilidad $\frac{\text{Nro. inserciones} + \text{Nro. eliminaciones}}{\text{Nro. reg. fichero}}$
- Tasa In/Out ratio $\frac{\text{Nro. accesos}}{\text{Nro. reg. accedidos}}$

Problemas ligados al diseño del Sistema Gestor de Ficheros:

P1) Los soportes físicos a utilizar pueden ser de distinto tipo y distintas características (independencia del dispositivo)

P2) El tiempo de acceso es, normalmente, excesivamente grande, debiéndose minimizar su efecto

P3) Tiene que servir para soportar un número de ficheros muy diverso (posiblemente grande), y de tamaños muy diferentes

P4) Gestión eficiente del espacio de almacenamiento

P5) Tiene que garantizarse a un nivel razonable la fiabilidad y seguridad de la información (frente a errores, fallos del sistema, y accesos)



Unidad 2: Operaciones entre archivos

Organización secuencial: Es la organización más simple (Se utiliza en las cintas magnéticas). Se basa en el acceso secuencial pero también puede optimizarse utilizando acceso relativo

- Registros Ordenados por identificador
 - Ventajas:

- Optimizar búsquedas: Elimina la necesidad de leer todo el archivo.
 - Procesamiento coordinado con otros archivos.
 - Cortes de control con un único recorrido
- Desventajas:
 - Problemas de inserción: implican un nuevo del archivo
 - Bajas costosas: Re-estructuración del archivo.
- Manipulación de registros:
 - Creación: Registros a cargar ya están validados y ordenados por identificador, ordenamiento externo de un archivo desordenado
 - Recuperación de registros: Consulta o recuperación unitaria. (Búsqueda lineal o binaria)
 - Reportes: Cortes de control(Registros ordenados por más de un campo)
- Actualización de Registros:
 - Inserción:(Directa) Crear un nuevo archivo, copiar los registros menores al insertar, luego agregar el registro nuevo, copiar el resto de los registros mayores, borrar el archivo viejo y renombrar el archivo nuevo con el nombre del viejo
 - Inserción:(Por novedades) Crear un nuevo archivo con las altas ordenadas; luego insertar por mezcla (Apareo de archivos).
 - Modificación: Se posiciona el registro por el identificador, desde el comienzo de la unidad y se reescribe.
 - Eliminación:(Directa) Crear un nuevo archivo, copiar los registros menores al insertar, copiar el resto de los registros mayores, borrar el archivo viejo y renombrar el archivo nuevo con el nombre del viejo.
 - Eliminación:(Por novedades) Crear un nuevo archivo con los registros a Eliminar; luego Eliminar por mezcla. (Apareo de archivos)
 - Eliminación:(Por marcado) Ubicar el registro por el identificador, Cambiar el signo del identificador o utilizar un campo de marcado (costo x bits). Cada tanto hay que depurar o compactar el archivo. (Mantenimiento).
- Registros Desordenados
 - Ventajas:
 - La inserción se hace al final del archivo sin ningún tipo de búsqueda
 - Desventajas:
 - Solo búsqueda lineal
 - No se puede coordinar procesamientos con otros archivos
 - No se pueden hacer cortes de control.
 - Manipulación de registros:
 - Creación: Creación y carga sin validación de unicidad ni búsqueda de espacio libre
 - Agregar al final: Apertura de archivo y posicionamiento al final para escribir (append)
 - Recuperación de registros: Posicionamiento al inicio del archivo y lectura secuencial de registros hasta encontrar el buscado o llegar al final del archivo. Solo búsqueda lineal.
 - Actualización de registros:

- Inserción: Se busca si el registro existe y si no existe se graba el nuevo buscando el espacio libre
- Modificación: Se recupera el registro buscado, se actualiza, se vuelve al comienzo de la unidad recién leída y escribe la unidad modificada
- Eliminación: (en RRLF) Se busca el registro a eliminar; se toma su posición; se graba el último registro del archivo en la posición a eliminar y trunca el archivo.
- Eliminación: (en RRLV) Se compacta el bloque donde se encontró el registro; se actualiza el espacio libre del bloque y se reescribe el bloque

Corte de control: Es un proceso en el cual partiendo de registros ordenados por uno o más campos, se los procesa en categorías determinadas por los criterios de ordenamiento. Se procesa un conjunto ordenado de registros en subconjuntos determinados por los criterios de orden.

Aplicaciones del Corte de Control:

La aplicación más común para la cuál se realiza el corte de control es para generar reportes que acumulen cantidades.

Se utilizan contadores, Sumadores, Promedios.

Se informan SubTotales por Niveles y Totales Generales

Apareo de Archivos: A partir de dos archivos (Uno principal y otro relacionado) y tienen alguna información que los enlace, generar un tercero como mezcla de los dos.



intersección, diferencia y unión: Flaco si no sabes esto volvé a álgebra A.

Nancy te espera♥

Ordenamiento de Archivos: Los algoritmos de búsqueda binaria, Fusión o apareo, corte de control, etc. Requieren de tener ordenados los datos.



Comparación de Algunos Métodos de Ordenamiento:

Algoritmo	Nombre	Complejidad	Memoria	Método	Estable
Burbuja	Bubblesort	$O(n^2)$	$O(1)$	Intercambio	Si
Inserción	Insertion sort	$O(n^2)$ "o menor"	$O(1)$	Inserción	Si
Urnas	Bucket sort	$O(n)$	$O(n)$		Si
Mezcla	Merge sort	$O(n \log n)$	$O(n)$	Mezcla	Si
Shell	Shell sort	$O(n^{1.25})$	$O(1)$	Inserción	No
Selección	Selection sort	$O(n^2)$	$O(1)$	Selección	No
Monticulos	Heapsort	$O(n \log n)$	$O(1)$	Selección	No
Rapido	Quick Sort	$O(n \log n), O(n^2)$	$O(\log n)$	Dividir	No

Burbuja:

Evalúa cada elemento de la lista con el siguiente intercambiandolos de posición si están en el orden equivocado $O(n^2)$.

Algoritmo:

```

COMIENZO
PARA j=1 HASTA N HACER
  PARA J=1 HASTA N-1 HACER
    SI dato[j] > dato[j+1] ENTONCES
      aux = datos[J]
      dato[j] = dato[j+1]
      dato[j+1] = aux
    FIN_SI
  FIN_PARA
FIN
  
```

```

COMIENZO
REPETIR
  ordenado = FALSE
  PARA j=1 HASTA N-1 HACER
    SI dato[j] > dato[j+1] ENTONCES
      aux = datos[j]
      dato[j] = dato[j+1]
      dato[j+1] = aux
    ordenado = FALSE
  FIN_SI
  FIN_PARA
  Hasta ordenado = TRUE
FIN
  
```

Inserción: Divide la lista en Parte Ordenada y Desordenada. Evalúa cada elemento de la Lista Desordenada y lo inserta en la lista ordenada $O(n^2)$ "o menor"

Algoritmo:


```

COMIENZO
PARA j=2 HASTA N HACER
    elemento = dato[j]
    j = j - 1
    MIENTRAS j >= 1 Y dato[j] > elemento HACER
        dato[j+1] = dato[j]
        j = j - 1
    FIN_MIENTRAS
    dato[j+1] = elemento
FIN_PARA
FIN

```

Ordenamiento por Urnas: Se divide el array en un conjunto de “Urnas” y cada urna se ordena por separado: $O(n^2)$. Bajo ciertas condiciones (Claves entre 0 y N-1 sin duplicados), la ordenación se puede hacer en $O(n)$:

Algoritmo:

Crear una colección de casilleros vacíos.

Colocar cada elemento a ordenar en un único casillero.

Ordenar individualmente cada casillero.

Devolver los elementos de cada casillero concatenados por orden.

Ordenamiento por Mezcla:

1. Si la longitud de la lista es 0 ó 1, entonces ya está ordenada. En otro caso:
2. Dividir la lista desordenada en dos sublistas de aproximadamente la mitad de tamaño.
3. Ordenar cada sublista recursivamente aplicando el ordenamiento por mezcla
4. Mezclar las dos sublistas en una sola lista ordenada
 $O(n \log(n))$

Algoritmo:

```

FUNCION Ordena_MergeSort(vec)
COMIENZO
    N = vec.length
    SI N <= 1 ENTONCES
        RETORNA vec
    SINO
        mitad = N/2
        CopiaVector(izq, 1, mitad)
        copiaVector(der, mitad+1, N)
        Ordena_MergeSort(izq)
        Ordena_MergeSort(der)
        mezcla(vec, izq, der);
    FIN_END
FIN {Funcion}

```

```

FUNCION Mezcla(vec, izq, der)
COMIENZO
    N = vec.length
    M = izq.length
    O = der.length
    i = 1 j = 1
    PARA k = 1 HASTA N HACER
        SI i >= M ENTONCES
            vec[k] = der[j]
            j = j + 1
        SINO SI j >= O ENTONCES
            vec[k] = izq[i]
            i = i + 1
        SINO
            vec[k] = der[j]
            j = j + 1
    FIN_END
FIN_PARA
FIN

```

Shell Sort:

Mejora al ordenamiento por inserción comparando elementos separados por un espacio de varias posiciones.

Generalmente se toma como intervalo inicial $n \div 2$. Luego se reduce los intervalos a la mitad hasta que el intervalo llegue a ser uno y aplica inserción. (Mejora el ordenamiento por inserción comparando elementos separados por un espacio de varias posiciones) $O(n^{1.25})$

Variante del algoritmo:

1. Aplicando intervalos de $N \div 2$
2. Aplicando Gonnet-Baeza-Yates $N*5 \div 11$.
3. Aplicando Steven Pigeon

Algoritmo:

Generalmente se toma como intervalo inicial $n \div 2$,

```
PROCEDURE Ordenar(A:LNaturales);
VAR i,j:longint;
    aux:qword;
    intv:longword; //intervalo
BEGIN
    intv:=length(A) DIV 2; //intervalo inicial
    WHILE intv>0 DO // Mientras intervalo >0
    BEGIN
        //algoritmo de inserción, por intervalos
        FOR i:=intv TO high(A) DO // Desde el intervalo hasta el final
        BEGIN
            aux:=A[i];
            j:=i-intv;
            WHILE ((j>=0) AND (aux<A[j])) DO
            BEGIN
                A[j+intv]:=A[j];
                j:=j-intv
            END;
            A[j+intv]:=aux
        END;
        //nuevo intervalo
        intv:=intv DIV 2
    END;
END;
```

Select sort:

Recorre todo el arreglo desde la primera posición, buscando el menor elemento de todos. Pone el Menor en la Primera posición. Repite todo el proceso para la sig posición y así sucesivamente $O(n^2)$

Quick Sort:

Permite, en promedio, ordenar n elementos en un tiempo proporcional a $n \log n$. Esta es la técnica de ordenamiento más rápida conocida.

Algoritmo:

1. Elegir un elemento de la lista, al que llamaremos pivote.
2. Situar los demás de la lista a cada lado del pivote, de forma que a un lado queden todos los menores que él, y al otro los mayores.
3. Cuando los índices se junten: Resituar el pivote.
4. Repetir este proceso de forma recursiva para cada sublista mientras éstas contengan más de un elemento.

Este algoritmo es inestable. No se adapta y es recursivo $O(n \log(n))$

Ordenamiento en disco: El ordenamiento externo se requiere cuando la información a procesar no cabe en la memoria principal de una computadora (RAM). Hay que utilizar memoria externa (un disco). El proceso es más lento.

Mezcla Natural (Natural merge sort)

El método consiste en aprovechar la existencia de secuencias ya ordenadas dentro de los datos de los archivos

Algoritmo:

1. A partir de las secuencias ordenadas existentes en el archivo, se obtienen particiones que se almacenan en dos archivos auxiliares.
2. Las particiones almacenadas en estos archivos auxiliares se aparean posteriormente para crear secuencias ordenadas.
3. 1 y 2 se repiten arbitrariamente hasta lograr ordenación de los datos contenidos en el archivo original

Se requieren para procesar, 2 Archivos como Mínimo.

Por seguridad se recomienda uno más. (3 ARCHIVOS)

Base de Datos

Usuarios de las DB:

- Programador de aplicaciones: Utiliza la DB con sus generando programas clientes
- Usuario final: Utiliza la Db por un programa Cliente
- Administrador de la base de datos: Responsable del control total del sistema

Componentes:

- Tablas: Archivo/s donde están los datos.
- Claves: Identificadores Primarios, Secundarios y Foraneos.
- Relaciones: Es la forma que se relacionan los datos.
- Funciones y Procedimientos: (Codigo Interno del Progrmador)
- Disparadores: Procesos que se ejecutan ante algun evento.
- Motores: Afectan al comportamiento de la DB.
- Su Organización interna y su método de acceso.
- Seguridad: (Manipulación de Usuarios y Permisos a distintos recursos y con distintos niveles)

Unidad 3 - Organización indexada: Árboles

Un árbol es una estructura jerárquica aplicada sobre una colección de elementos u objetos llamados nodos. Uno de estos nodos es la **raíz** del árbol.

Los árboles representan estructuras **no lineales y dinámicas**.

- **No lineal:** A cada nodo del árbol le pueden seguir otros varios.
- **Dinámica:** El árbol puede variar en la ejecución del programa.

Los árboles se pueden representar como: Diagrama de Venn, anidación de paréntesis, notación decimal, notación indentada y grafos.

Características y propiedades de árboles

1. Todo árbol no vacío tiene **un solo** nodo raíz.
2. Nodo X descendiente de nodo Y == X hijo de Y (X rica e Y boke).
3. Nodo X antecesor directo de nodo Y == X padre de Y.
4. Los hijos de un nodo son hermanos entre sí.
5. Nodo sin hijos == Nodo terminal (hoja).
6. Nodo que no es raíz ni hoja == nodo interior.
7. Grado de nodo: Número de hijos de un nodo.
8. Grado de un árbol: Máximo grado entre todos los nodos.
9. Nivel: Comienza en 0 (en la raíz).
10. Altura: Nivel + 1

Todo al revés los culiados estos

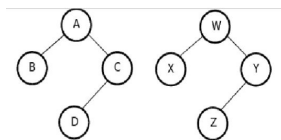
Árbol binario

Árbol donde cada nodo puede tener como máximo dos hijos. (Grado del árbol = 2)

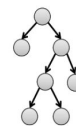
Tipos de árboles binarios:

- **Árboles binarios similares:** Estructuras idénticas (Misma organización de nodos). La información de los nodos varía.
- **Árboles binarios distintos:** Estructuras diferentes.
- **Árboles binarios equivalentes:** Estructuras idénticas (similares) y con la misma información en los nodos.

**Al hablar de estructura me refiero a cómo están distribuidos los nodos.

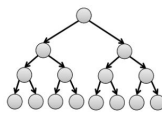


Estos son árboles binarios similares (los nodos son distintos).

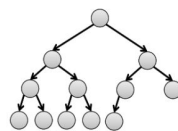


- **Árbol estricto:** Cada nodo puede tener 0 o 2 hijos.
- **Árbol lleno:** Árbol estricto donde la altura del subárbol izquierdo es la misma que el

derecho (y ambos árboles llenos).



- **Árbol completo:** Árbol lleno hasta el penúltimo nivel. En el último nivel, los nodos



están agrupados a la izquierda.

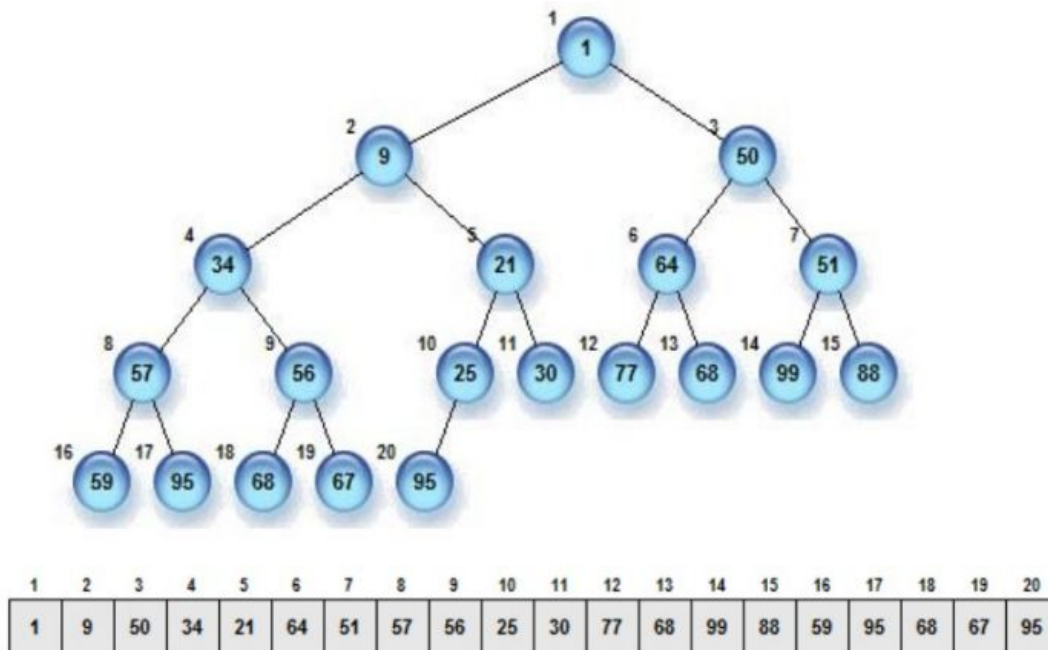
Las **desventajas** de los árboles binarios son:

- Suelen tener muchos punteros a NULL (memoria dinámica) o muchas componentes con valor -1 (memoria estática).
- Si un árbol tiene n nodos, existe 2n enlaces, donde n+1 son enlaces vacíos.

- El recorrido con programas recursivos resulta costoso ya que implica un gasto adicional de memoria y tiempo de ejecución.

Montículos binarios

Árbol completo cuyos nodos almacenan elementos comparables mediante $\leq$ y todos cumplen la propiedad de montículo (todo nodo es menor que sus descendientes (montículo de mínimos)).



Características y propiedades de montículos

- **Nodo raíz** (primero del vector) es el **mínimo**.
- **Altura** de un montículo **logarítmica** respecto al número de elementos almacenados.
- Si un elemento no cumple con la propiedad de montículo, se pueden realizar **ascensos** (intercambio con su padre) o **descensos** (intercambio con el mayor de sus hijos). La cantidad de operaciones es proporcional a la **altura**.
- Para **insertar** un nuevo elemento, **se coloca al final del vector y se asciende** hasta cumplir la propiedad de montículo.
- Para **eliminar raíz** se intercambia con el último elemento y se **desciende** la nueva raíz hasta cumplir propiedad.

Utilización de TDA Cola.

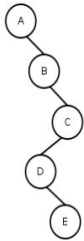
- **Acceso al mínimo:** orden 1 ($O(1)$).
- **Inserción por valor:** orden $\log n$ ($O(\log n)$).
- **Borrado de mínimo:** orden $\log n$ ($O(\log n)$).
- $O(n)$ para la **creación a partir de un vector**. (No utiliza memoria dinámica)
- **Borrado o modificación de elemento sabiendo su posición:** orden $\log n$ ($O(\log n)$).

Desventajas:

- **Búsqueda y acceso al i-esimo menor** se comporta igual que un vector desordenado.
- **Fusión de montículos binarios:** orden n ($O(n)$)

Arboles degenerados (los que le gustan a luqui)

Árbol donde cada nodo tiene un solo hijo. Se comporta como una lista, perdiendo la ventaja de eficiencia de un árbol con respecto a una lista.



Dos formas de **representar** un árbol binario:

1. Arreglos o listas enlazadas.
2. Datos tipos punteros (variables dinámicas).

Donde el $2i+1$ Hijo Izquierdo y $2i+2$ hijo Derecho

La forma sería mediante un nodo que tiene dos punteros (izq y derecha) y un atributo de información.

Recorridos de arboles binarios

Recorrer un árbol significa visitar los nodos de manera sistemática, de manera que sean visitados todos una sola vez.

Existen distintas formas de recorrer un árbol:

1. **En anchura (a lo ancho - por niveles):** Mostrar todos los elementos del mismo nivel, y continuar hasta el último.
2. **En profundidad:**
 - a. **Preorden:** Raíz - izq - der.
 - b. **Inorden:** Izq - raíz - der.
 - c. **Postorden:** Izq - der - raíz.

Árbol de búsqueda binario

Árbol que tiene al menos un nodo y cumple con la condición de que cada hijo a izquierda es menor a su padre, y cada hijo a derecha es mayor a su padre.

Para la utilización de este tipo de árbol es necesario que los elementos que insertamos sean comparables.

Características y propiedades de árbol de búsqueda binario

- Árbol binario cuyos nodos almacenan elementos comparables mediante $\leq$ y todo nodo cumple **propiedad de ordenación** (Todo nodo es mayor que los nodos del subárbol izquierda y menor que los nodos del subárbol derecho)
- **Básicamente lo que puse al describir un ABB.**
- El **recorrido inorden** recorre (xd) los elementos en orden de menor a mayor.
- El elemento **mínimo** es el hijo más a la izquierda del árbol.
- El elemento **máximo** es el hijo más a la derecha del árbol.
- Para **buscar** un valor se parte de la raíz y se desciende realizando comparaciones (va a izquierda si es menor o a derecha si es mayor) hasta hallar el valor.
- Para **insertar** un elemento se coloca siempre como hoja en la posición que corresponde.
- Para **eliminar** un elemento se enlazan con el padre su hijo (si tiene 1). Si tiene 2 hijos, se intercambia con el máximo del subárbol izquierdo y se borra el máximo.

Órdenes de operaciones de un ABB

- **Acceso por valor (búsqueda):** orden h ($O(h)$).
- **Insertión por valor:** orden h ($O(h)$).
- **Borrado por valor:** orden h ($O(h)$).

- **Acceso al i-esimo menor:** orden h ($O(h)$).
- **Borrado del i-esimo menor:** orden h ($O(h)$).
- **Fusión:** orden n ($O(n)$).

** h = altura; n = cantidad de nodos.

Se denomina **árbol equilibrado** al que garantiza que su altura es logarítmica (h está entre 0 y $O(\log n)$)

Los árboles BB **no son equilibrados**.

Árbol balanceado

Árbol que intenta mantener la menor profundidad en sus dos subárboles.

El balanceo o equilibrio hace que algunas operaciones sean más eficientes, en especial las búsquedas.

El inconveniente de los árboles binarios es que se desbalancean fácilmente.

Árbol AVL

Árbol binario de búsqueda (ABB) que siempre está balanceado. Intenta mantener la menor profundidad en sus dos subárboles. Cuando se realiza inserción o eliminación, se comprueba si el árbol está desequilibrado y se realiza el balanceo.

Características y propiedades de un AVL

- Todo AVL tiene **altura logarítmica**.
- **Árbol de Fibonacci (F_h):**
 - F_{-1} : árbol vacío.
 - F_0 : árbol con único nodo
 - F_h árbol con subárbol izquierdo F_{h-2} y derecho F_{h-1}
 - Altura h y número de elementos:

$$N(h) = N(h-1) + N(h-2) + 1$$

$$N(h) \in O(\phi^h) \Rightarrow h \in O(\log n)$$

- Un árbol de Fibonacci es un AVL con mayor desequilibrio.
- Un árbol binario es un AVL si y sólo si cada uno de sus nodos tiene un equilibrio de -1, 0 o +1 (progra 2)
- Rotaciones si no se cumple lo de arriba.
- Rotaciones posibles:
 - Rotación simple a la derecha (RSD)
 - Rotación simple a la izquierda (RSI)
 - Rotación doble a la derecha (RDD)
 - Rotación doble a la izquierda (RDI)

Orden de búsqueda de binario: $\log_2(N+1)$

Orden de búsqueda de un AVL: $1.44 * \log_2(N+2)$

El tiempo de acceso de los sistemas de almacenamiento masivo suele ser mucho mayor que el tiempo de transferencia (se tarda mucho más en encontrar y acceder al dato que se busca que transferirlo).

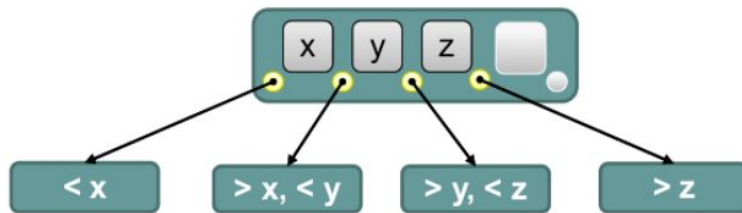
Para grandes volúmenes de datos sería **conveniente reducir el número de accesos**, a cambio de que esos accesos **contuvieran elementos de mayor tamaño**.

Árboles (a,b)

Los **árboles (a,b)** son árboles generales (no binarios) donde cada nodo interno puede tener un número de hijos $m+1$ en el rango de $[a,b]$.

Cada nodo almacena m **claves** (elementos comparables), **ordenadas de menor a mayor**.

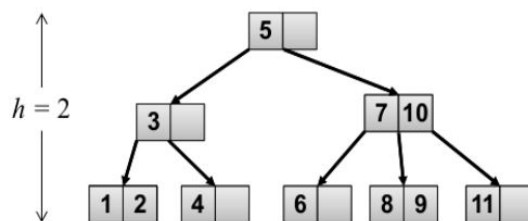
Sirven para que se pueda utilizar como un **árbol de búsqueda**.



Árboles B

Un árbol B de **orden d** es un árbol $(d+1, 2d+1)$ con las siguientes propiedades:

- La **raíz** puede tener **cualquier** número de claves.
- Todas las hojas se encuentran en la **misma profundidad h**.
- Es un árbol equilibrado (por lo de arriba). Su altura es logarítmica respecto al número n de claves almacenadas.
- **Ejemplo:** Un árbol B de orden 1 es un árbol (2,3): Cada nodo puede contener 1 o 2 claves y tener 2 o 3 hijos.



Características y propiedades de un árbol B

- **Búsqueda:**
 - Se desciende desde la raíz hasta el nodo que contenga al elemento.
 - En cada nodo se busca en el array de claves (búsqueda secuencial o binaria). Si no se encuentra, se pasa al hijo que coincida con el rango de valores que se busca (Foto de arboles (a,b))
- **Inserción:**
 - Se desciende igual que en búsqueda (hasta el nodo hoja que debería contener el elemento).
 - Se inserta en la posición adecuada del array de claves del nodo.
 - Si se supera en el número máximo de claves, se genera reestructuraciónReestructuraciones posibles:
Alta paja explicar cada uno, fíjense en
Clase_03_Archivos_Indexados_Arboles_parte2.pdf (empieza pag 10)
 - División
 - Transferencia
 - Fusión
- **Borrado:**
 - **Borrado en nodo interno:**

- Se desciende desde la raíz hasta el nodo que contenga el elemento a borrar.
- Se intercambia con el máximo del hijo izquierdo o con el mínimo del hijo derecho (se elige el hijo con más claves).
- Se borra el elemento en el hijo.
- **Borrado en nodo hoja:**
 - Se elimina del array de claves.
 - Si con la eliminación, el número de claves es $d-1$, entonces se genera reestructuración.

Conclusiones de árboles B

1. La letra B no es binario.
2. Los árboles B **NUNCA** son binarios.
3. No son árboles de búsqueda.
4. No es porque sean balanceados, ya que suelen no serlo.

Árboles B+

Árbol donde solo las hojas contienen elementos. Los nodos internos contienen claves para dirigir la búsqueda. Los nodo hoja forman una lista doblemente enlazada.

Características y propiedades de un árbol B+

- Tiene un límite máximo y mínimo en el número de claves por nodo.
- Este tipo de árbol es una variación de un árbol B.
- Toda la información se guarda en las hojas.
- Los nodos internos solo contienen claves y punteros.
- Todas las hojas se encuentran en el nivel más bajo.
- Permite la recuperación en rango mediante búsqueda secuencial.
- Conservan la propiedad de acceso aleatoria rápido (de los árboles B) permitiendo además un recorrido secuencial más rápido.
- Ocupan más espacio que los árboles B (ya que algunas claves se repiten).
- Cada nodo tiene máximo M descendientes.
- La raíz tiene al menos dos descendientes (o ninguno)
- Un nodo que no sea hoja, que tiene K descendientes contiene $k-1$ llaves.
- Los nodos internos no tienen datos sino punteros a los datos (ya lo puse 8 veces pero para que se entienda).

Está compuesto por dos partes:

1. **Índice:** nodos interiores.
2. **Secuencia:** hojas enlazadas secuencialmente en la que se repiten las claves anteriores.

Operaciones

- **Búsqueda:** La búsqueda no debe detenerse cuando se encuentra la clave en la raíz o nodo interior, sino seguir hasta llegar al nodo hoja que lo contiene.
- **Inserción:** Proceso idéntico al de árboles B, salvo cuando se intenta insertar en un nodo interno que está lleno. Cuando esto sucede, hay que reestructurar.
- **Eliminación:** Más fácil que árboles B debido a que las claves se encuentran en las hojas.

Si al eliminar la clave, la cantidad de claves es mayor o igual a $m/2$, termina el proceso. Cuando esto sucede, hay que reestructurar.

Resumen de comparación entre B y B+:

Arboles B+ y B:

• Comparaciones

	Arbol B	Arbol B+
Ubicación de datos	Nodos (cualquiera)	Nodo terminal
Tiempo de búsqueda	=	=
Procesamiento secuencial	Lento (complejo)	Rápido (con punteros)

Arboles B*

Árboles con las siguientes características:

- Cada nodo tiene un máximo de M descendientes.
- Cada nodo excepto el raíz tiene al menos $(2m-1)/3$ hijos.
- La raíz tiene al menos dos descendientes (a menos que sea una hoja).
- Una hoja contiene al menos $E[(2m-1)/3]$ claves, y no más de $m-1$.
- Todas las hojas aparecen en el mismo nivel.
- Un nodo que no sea hoja con k descendientes contiene $k-1$ llaves.

Los lexicográficos y Patricia están al final del pdf xd

Unidad 4 - Organización directa: Hashing

Algoritmos de dispersión (hashing)

La organización indexada se basa en mantener un índice (claves + referencias) para que la búsqueda sea con el índice para acceder al dato. El objetivo de este tipo de organización es el de reducir al máximo el coste de las operaciones de búsqueda.

La solución se da en el hashing, diseñando una función de cálculo para transformar la clave en una referencia física.

En esto se basa **la organización dispersa**:

- No se almacenan las claves en un índice.
- El acceso al dato se da a través de una función de dispersión aplicada a la clave.

El **hashing** es una técnica que se utiliza para generar una dirección base única para una clave dada. **Se utiliza cuando se requiere un acceso rápido a un registro.**

Las operaciones de insertar, buscar y eliminar en tablas de direccionamiento directo (arreglo de 0 a n) son de orden 1 ($O(1)$).

Algunas características

- Las tablas hashing son tablas que relacionan claves con posiciones de memoria donde se almacenan atributos.

- Se aplica cuando el conjunto de claves es mucho mayor al número de claves reales a almacenar.
- **Desbordamiento:** Ocurre cuando una nueva clave se aplica a una dirección de memoria completamente ocupada.
- **Colisión:** Ocurre cuando dos claves distintas se aplican sobre la misma celda.
- **Densidad de claves:** Cociente entre número de claves en uso m y el número total de llaves posibles n . $\rightarrow \frac{m}{n}$
- **Factor de carga o Densidad de carga α :** Cociente entre el número de claves en uso m y el número total de registros almacenados en la tabla de dispersión.
 $\alpha = \frac{m}{s*b}$ siendo:
 -s = número de registros por bloque.
 -b = número de bloques que hay en la tabla de dispersión

Funciones hash

Características

- Fáciles de calcular.
- El valor que calcula depende de la clave.
- Producir el menor número de condiciones ya que hace que la ocupación de los bloques sea equiprobable.

Problemas

- Funciones uniformes que dividen el conjunto de claves posibles en grupos de claves exactamente iguales. (hashing muy chico).

Métodos de construcción de un hashing

1. **Resto de división o Módulo:** La clave se divide entre el número de direcciones (se trata de que sea un **número primo** ya que tiende a distribuir residuos en forma más eficiente.) El número primo es divisible por sí mismo, lo cual no va a generar colisión que dos claves que sean divisibles por el mismo número.
Espacio de direcciones: $[0, M-1]$, siendo M el primo elegido para el mod.
2. **Multiplicación:** Se multiplica la llave por una constante, y luego se multiplica por m . La constante A se encuentra en el rango $0 < A < 1$.
La clave k se multiplica por la constante A , se extrae la parte fraccionaria de la multiplicación, luego a esa parte fraccionaria se la multiplica por m y se toma el piso del resultado.

La función hash es:

$$h(k) = m(kA \bmod 1)$$

La ventaja de este método es que el valor de m no es crítico.

Generalmente este valor m es una potencia de 2.

A pesar de que funciona para cualquier valor de A , se ha encontrado que funciona mejor con algunos valores como $A \approx 0.6180339887...$

3. **Plegado FOLD & ADD:** Se divide la clave en partes iguales y se suman. La suma de las partes puede realizarse de dos formas:
 - a. **Por desplazamiento:** Se suman las partes directamente.
 - b. **Por fronteras:** Se pliega el identificador por las fronteras de las partes y se suman los dígitos coincidentes.

4. **Compresión:** Dividir la clave en componentes, traducir su número de cotejo a binario, aplicar XOR y al resultado la operación resto de división entera. (Hecho en TP4 punto2).
5. **Extracción:** Método de la mitad del cuadrado, donde a la clave numérica se la eleva al cuadrado y nos quedamos con algún número de la zona central del resultado. La cantidad de dígitos que tomamos del resultado es equivalente a los que necesitamos para mapear el array.

El hashing perfecto se da cuando no se producen colisiones. El imperfecto/imperfecto, lo contrario xd.

Soluciones para colisiones

1. **Propagar registros:** Buscar funciones que distribuyan aleatoriamente los registros para evitar agrupaciones de llaves que produzcan las mismas direcciones.
2. **Usar memoria extra:** Poner más espacio de direcciones posibles que registros a utilizar. (esto es de trolol).
3. **Colocar más de un registro en una dirección:** Concepto que se basa en **cubetas** de datos en cada dirección, donde ahí se colocan los registros que colisionan de manera que al hacer una búsqueda podamos recuperar la cubeta entera y ahí buscar el registro deseado.
No sé si lo que viene abajo hace referencia a la probabilidad de colisión.
4. **Sondeo lineal:** Intervalo entre cada intento es constante (frecuentemente 1)
5. **Sondeo cuadrático:** Intervalo entre intentos aumenta linealmente.
6. **Doble hash:** Intervalo de intentos es constante para cada registro, pero es calculado por otra función hash.

Métodos para la resolución de colisiones

1. **Progressive Overflow:** Se obtiene la dirección con la función hash. Si la dirección es libre, entonces se coloca el registro ahí. Si no es así, entonces se busca la primer dirección consecuente que esté libre y se inserta.
Si se llega al final del archivo, comenzar desde el principio.
Este caso es el que se hizo en el TP4 ej1 de programación.
2. **Distribución en el almacenamiento por cubetas:** Se asignan bloques de espacio (cubetas) para almacenar múltiples registros, en vez de asignar celdas individuales para cada uno.
Cuando una cubeta es desbordada (no admite más registros), una nueva cubeta deberá ser asignada para el registro.
Tiene los mismos problemas que las colisiones por celda.
Si el direccionamiento abierto es utilizado, el espacio disponible se busca en la siguiente cubeta.
Se tiene que minimizar la cantidad de claves por cubeta.
3. **Doble Hashing:** Técnica basada en generar otro hash, quizás con una fórmula diferente.
4. **Encadenado de Progressive Overflow:** Se basa en crear una lista encadenada entre los distintos registros sinónimos (colisiones)
5. **Encadenado con Separate Overflow Área:** Se emplea el concepto de encadenamiento a través de una lista encadenada, pero las colisiones se almacenan en otra área de overflow.

En el archivo de datos solo se almacena la cabeza de dicha lista.

Este caso es el que se hizo en el TP4 ej2 de programación.

6. **Scatter Tables (tablas dispersas):** No existe área primaria de datos, solo apuntadores hacia distintas cabeceras de listas donde residen aquellas llaves sinónimas (colisiones).
Existen distintas implementaciones, ya sea usando muchos archivos, listas o archivos con listas de listas.
7. **Hash Extendido:** Permite que el archivo crezca sin cambiar la función de hashing.
Para este método se necesitan cuatro estructuras:
 - a. **Archivo de tabla de dispersión:** Guarda tamaño de tabla (tt). La tabla posee punteros a bloques.
 - b. **Archivo con bloques:** Cada bloque soporta cantidad fija de registros de longitud fija. Cada bloque tiene su tamaño de dispersión (td).
$$\text{bloque.td} = ((\text{tabla.tt}) / \# \text{ref s bloque en tabla}) = \text{distancia en tabla a otra ref}$$
 - c. **Archivo con lista de bloques libres.**
 - d. **Función de hashing.**

Análisis de tablas Hash

- Hash tradicional eficiente pero con dos problemas: colisiones y el hecho de ser estático.
- El hash extendido es una estructura relativamente moderna que promete ser mejor que el B-Tree pero lo limitan algunas cosas:
 - No se tienen algunas métricas importantes acerca de la capacidad de cada cubeta.
 - La estructura crece dinámicamente y permite búsqueda de alta velocidad, pero tiene acceso secuencial.

Unidad 5 - Índices e information retrieval

Todo lo que se habla en esta primera parte es para organizaciones indexadas.

Árbol hilvanado

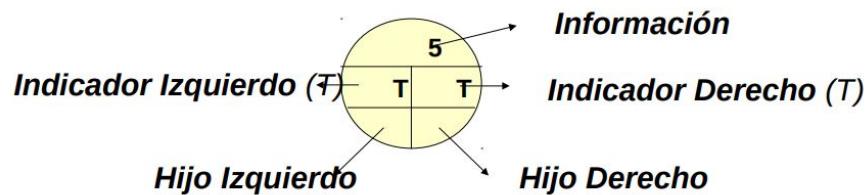
Árbol binario en el que cada hijo izquierdo de valor nulo es sustituido por un enlace al nodo que le antecede en el orden simétrico, y cada hijo derecho de valor nulo es sustituido por un enlace al nodo que él sigue en el recorrido en orden simétrico.

Estos enlaces que cumplen una función nueva y especial se llaman HILVANES.

- Si un nodo tiene un hijo derecho vacío, este enlace será un hilván que apuntará al nodo siguiente según el recorrido que se esté realizando (ej: inorden).
- Si un nodo tiene un hijo izquierda vacío, este enlace será un hilván que apuntará al nodo anterior según el recorrido que se esté realizando (ej: preorden).

Cada nodo del árbol hilvanado contiene:

1. Referencia a su información.
2. Puntero a hijo izquierdo.
3. Indicador izquierdo (verdadero o falso).
4. Puntero a hijo derecho.
5. Indicador derecho (verdadero o falso).



Multilistas

- **Listas enlazadas simples:** Se pueden implementar índices por medios de listas simplemente enlazadas.
 - Se necesitan dos áreas, una para datos y una para índices enlazados.
 - Las listas simplemente enlazadas tienen un puntero al próximo nodo.
- **Listas enlazadas dobles:**
 - Se necesitan dos áreas, una para datos y otra para índices enlazados.
 - Cada elemento de la lista tiene un puntero al anterior y al próximo nodo.
- **Listas enlazadas circulares:**
 - Se necesitan dos áreas, una para datos y otra para índices enlazados.
 - El último de la lista apunta al primero.
- **Listas multilistas:**
 - Permite acceder a la información de manera ordenada a través de campos claves.
 - Permiten llegar a un registro por diferentes caminos.
 - El camino lo determina la clave por la que se haga la búsqueda.
 - **Las multilistas son recomendables** cuando la búsqueda se hace **con solo un atributo**.
 - **Es preferible usar listas invertidas** en caso de necesitar una **combinación de atributos**.
- **Entidades y relaciones:**
 - **Entidad:** Representa una "cosa", "objeto" o "concepto" del mundo real con existencia independiente.
Ejemplos: Persona, automóvil, casa.
Está descrita y se representa por sus características o atributos.

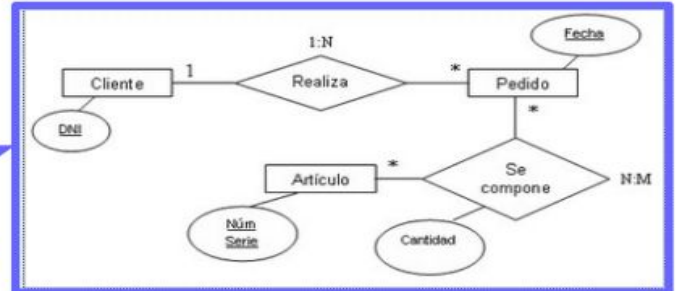
- **Relación:** Asociación o relación matemática entre varias Entidades.

■ **Por ejemplo:**

■ El **Ciente Realiza Pedido**.

■ El **Pedido Se Compone** de **Articulo**

Modelo ENTIDAD-RELACIÓN



■ **Cardinalidad:** Cómo se relacionan las entidades:

- **Uno a uno: (1:1)** Un registro de una entidad A se relaciona solo con un registro de una entidad B.
- **Uno a varios: (1:N)** Un registro en una entidad en A se relaciona con cero o con muchos registros de una entidad B. Los registros de B solo se relacionan con un registro en A.
- **Varios a uno: (N:1)** Una entidad en A se relaciona exclusivamente con una entidad de B. Una entidad en B se puede relacionar con 0 o muchas entidades en A.
- **Varios a varios: (N:M)** Una entidad en A se puede relacionar en 0 o con muchas entidades B o viceversa.

● **Implementación:**

Se requiere al menos:

- Una estructura para Entidad A.
- Una estructura para Entidad B.
- Una estructura para la relación A-B.
- Una estructura Multilista que los engloba.

● **Implementación en archivos:**

Se requiere al menos:

- **Archivo cabecera:**
 - Almacena las direcciones de inicio de cada cadena de características.
 - Recomendablemente incluye un campo de longitud de cadena que permite seleccionar el acceso de la cadena más corta cuando se conocen dos o más características.
 - Tantos registros como características tenga el modelo.
- **Archivo Principal (Datos):**
 - Además de los campos de almacenamiento normales, utilizan tantos campos adicionales como atributos tenga el modelo.
 - Cada uno de estos será una conexión al siguiente registro con la misma característica.

Esto que describí antes lo hicimos en el TP5 - ej2.

Índices invertidos

Es conocido como archivo de anuncios o archivo invertido. Se basa en una estructura de datos de índice de una asignación de contenido, a sus localizaciones en un archivo de base de datos, o en un documento o conjunto de documentos.

Características de índices invertidos

- Permite **búsquedas rápidas de texto completo** a un buen costo de procesamiento cuando un documento se agrega a la base de datos.
- Es la estructura de datos más popular utilizada en **recuperación de documentos sistemas**.
- Utilizado a gran escala en **motores de búsqueda**.
- Dos variantes principales:
 - **A nivel récord invertida índice:** Contiene una lista de referencias a los documentos de cada palabra.
 - **A nivel de palabra invertida índice:** Contiene las posiciones de cada palabra dentro de un documento.
Esta forma ofrece más funcionalidad pero necesita más potencia de procesamiento y espacio para crear.

Índices:

Implementación en archivos:

- Los **índices** son **archivos de control** que facilitan la recuperación de registros de datos.
- Los **registros de índice** se componen del valor de una **clave de recuperación** de registros de datos, y de una o más **referencias a los registros de datos** que corresponden a ese valor.
- Las **claves de recuperación** de registro de datos se definen a partir de **uno o más atributos de registros** de un archivo de datos.
- Una **clave de recuperación** puede implicar la realización de **algún cálculo sobre uno, alguno o todos los atributos que involucra**.
- Las **claves de recuperación** pueden ser **atómicas o compuestas**.
- Las **referencias a registros de datos** pueden ser **posiciones de unidades en el archivo o valores de un identificador**.

Ventajas:

1. Menor **tamaño** de índice implica **mayor eficiencia de búsqueda**.
2. Registros del **índice de longitud fija** implica **acceso más simple y eficiente**.
3. Simplifica la utilización de registros de longitud variable.
4. Permite **acceso directo** (eficiente) a los datos utilizando cualquier campo clave. Incluso **simultáneamente** (varios índices).
5. Permite **mantener** (el mantenimiento del orden es muy eficiente) una ordenación lógica sin moverlos.

Inconvenientes:

- Son más lentas las operaciones de procesamiento secuencial (afectan muchos datos).

Tipos de índices:

- **Índice primario:** Se utiliza la clave primaria del dato
- **Índice de agrupamiento:** Se utiliza un campo de ordenación del dato
- **Índice secundario:** Se utiliza cualquier otro campo (clave o no).
- **Índice multinivel:** Si el índice es muy grande, se aplica un índice sobre el índice.

Aumenta la eficacia de búsqueda pero complica la gestión.

Organizaciones con varios índices secundarios: Archivos invertidos

Problemas

- Los índices secundarios tienen **valores repetidos** (salvo que sean claves).
- La actualización/inserción/eliminación implica re-actualizar índices.
- Para el acceso a través del índice secundario hay que realizar una inversión del proceso de obtención de información

Archivo invertido: Archivo organizado a partir de índices secundarios basados en la clave primaria.

Invertido con respecto a la clave primaria.

Existen dos tipos:

1. Totalmente invertidos (se utilizan todos los campos).
2. Parcialmente invertido (solo se utilizan algunos campos).

Ventajas

- La información del índice no tiene por que estar en el dato (es redundante).
- Acceso eficiente para diferentes criterios (distintos campos).
- Se pueden responder a preguntas complejas sin acceder a los datos (operaciones con listas).

Referencia a los datos

Utilización de la clave primaria como referencia.

Ventajas

- Mayor fiabilidad.
- No es necesario reorganizar el índice secundario en la eliminación.
- Mayor independencia en la ubicación de los datos.

Problemas

- Mayor coste en la búsqueda (hay que buscar en el índice primario).

Organización B+ y Organización Indexada

Objetivo: aprovechar las ventajas de las organizaciones

- * indexada → búsqueda
- * secuencial → trat. secuencia

Idea básica: árbol (→ árbol B) sobre **una secuencia** o de las organizaciones

Árbol B+ :

- Toda información (claves o datos) está en un **conjunto encadenado de bloques (hojas)** → **conjunto secuencia**
- El árbol de acceso (→ árbol B) está formado con **copia de las llaves** (pueden estar repetidas), que actúan como separadores → **conjunto índice**.
- Tratamiento un poco diferente del árbol B (en las hojas)
- La altura es algo menor que para un árbol B

La organización secuencial indexada = Organización basada en un árbol B

Si alguien entiende algo de esa diapositiva great

Motores de búsqueda

Es un sistema que:

- Construye un índice a partir de un texto.
- Responde a consultas utilizando este índice.

Y ofrece:

- Escalabilidad
- Ranking de relevancia
- Integra diferentes fuentes de datos
- Trabaja con archivos invertidos (utiliza índices invertidos).

Proceso de indexación:

1. Convertir el documento.
2. Extraer texto y metadatos.
3. Normalizar el texto.
4. Crear el índice invertido.

Esto lo hace evaluando palabras, no substrings.

La estructura del archivo inverso es la siguiente (guiado por ejemplo de pdf):

- **Diccionario:** Conjunto de todas las palabras distintas que aparecen en el texto.
- **Ocurrencias (postings):** listas conteniendo toda la información necesaria para cada palabra del vocabulario (documentos donde aparece, posición, frecuencia, etc).

Pasos para la construcción de un motor de búsqueda:

1. **Parseo:** A cada documento se le extraen los términos (palabras) y el ID documento (en que documento está).
2. **Ordenamiento:** Se ordena el archivo por términos (palabras), manteniendo duplicados.
3. **Agrupamiento:** Corte de control por término y documento, agrupando términos duplicados y marcando su frecuencia para cada ID documento
4. **Generación de Diccionario y Posting:** Se genera el diccionario y posting.
 - a. **Diccionario** (Archivo inverso con índice no denso):
 - i. Agrupa por término.
 - ii. Almacena la frecuencia total
 - iii. Direcciona al Posting.
 - b. **Posting** (Archivo con lista inversa):
 - i. Direcciona (ubicación de los documentos, url, etc).
 - ii. Almacena frecuencia de cada documento.

Diccionario y Posting explicado arriba de los pasos.

Búsquedas sobre el archivo invertido

La búsqueda más simple se da con una única palabra. Estas se realizan utilizando una estructura de datos adecuada (ej: Hashtables, trees, b-trees, etc).

Las consultas **conjuntivas (and)** implican:

- Buscar todas las palabras de la consulta.
- Forma más popular en la web.
- Requiere la intersección de todas las listas invertidas

Las consultas **disyuntivas (or)** implican:

- La unión de las listas.

Consultas contextuales:

- Son más difíciles de resolver con índices invertidos.
- Todas las palabras aparecen en secuencia (una frase).
- Aparecen lo suficientemente (por proximidad).

- Las soluciones incluyen indexación de pares de palabras o índices posicionales, incrementando tamaño de índice.
- Las búsquedas por prefijo o por rango son básicamente consultas disyuntivas mayores.
- Para las búsquedas de expresiones regulares se requiere una búsqueda secuencial.

Recuperación de textos

Existen distintas variantes de búsquedas:

❖ Modelo booleano

- **Conjunción (and, +):** Los documentos recuperados deben tener **todos** los términos de la búsqueda.
- **Disyunción (or, -):** Las unidades de recuperación (documentos) deben contener **alguno** de los términos específicos.
- **Exclusión (not):** Los documentos **no deben tener** los términos especificados.

❖ Características modelo booleano:

- Primer modelo implementado y más usado.
- Simple de implementar.
- Implementaciones eficientes.
- Más utilizado en administración de bases de datos.
- Los resultados de las búsquedas tienen las mismas relevancias.
- No contempla el cumplimiento parcial.
- No contempla la cantidad de veces que una palabra se encuentra.

❖ Modelo vectorial

- **Conjunto de documentos (D):** $D = \{d1, d2, d3, \dots, dn\}$
- **Conjunto de todos los términos relevantes del conjunto D:**
 $T = \{t1, t2, t3, \dots, tn\}$
- **Se modela vectorialmente.**

❖ Características modelo vectorial:

- El término tiene un peso en el documento.
- Cuanto mayor frecuencia, más peso tiene.
- Ley de Zipf: Puede haber alta frecuencia de términos globales y baja frecuencia en cada documento.
- Los vectores se normalizan para neutralizar los tamaños de los documentos con los términos.
- Permite ordenar documentos por relevancia.
- Permite agrupar documentos similares.
- Similitud: 2 documentos iguales -> Similitud = 1
2 documentos distintos -> Similitud = 0

❖ Consultas especiales:

- **Frases textuales:** No basta si los términos están en el documento. Hay que saber además sus posiciones relativas en el mismo.
- **Términos próximos:** Son para buscar frases equivalentes. Se puede pedir que los términos estén a menos de una distancia. Hay que conocer la ubicación relativa de los términos en el documento
- **Patrones:**
 - Utilización de comodines (* y ?).
 - Expresiones regulares (ej: Sr. Sra Martinez).

- **Términos parecidos:**
 - No se está seguro cómo se escribe.
- ❖ **Índices invertidos:**
 - Se basan en la clasificación por términos.
 - El conjunto de términos de un documento se los llama:
 - Vocabulario o léxico.
 - Las listas de términos se las llama invertidas.
- ❖ **Modelo Booleano:** Se organizan en orden creciente de documento para optimizar operaciones de conjunto.
- ❖ **Modelo Vectorial:** Cada elemento de la lista invertida de términos es para cada documento que aparezca.
- ❖ **Para Búsqueda de Frases y términos:** Para cada término se debe tener la lista de posiciones donde aparece en el documento.
- ❖ **Índices de firmas:**
 - La firma de un documento sirve para verificar si un término está contenido.
 - Solo corresponde al modelo booleano.
 - Para un tamaño X de bits de la firma se aplica hashing entre 0 y X-1 a los términos.
 - Se almacenan en un archivo secuencial con registros correspondientes a la firma de cada documento.
 - La búsqueda en el archivo de firmas es secuencial.

Distancia de Edición de Levenshtein: Número mínimo de operaciones requeridas para transformar una cadena de caracteres en otra (se usa ampliamente).

Heurísticas

La heurística se basa en la **búsqueda con información**, lo opuesto a la heurística sería una búsqueda en anchura o en profundidad que debe recorrer todos los nodos “a ciegas”. Es una estrategia que limita grandiosamente la búsqueda de soluciones ante grandes espacios de problemas.

Algunos métodos de búsqueda son:

- **Búsqueda por Escalada simple o Hill Climbing (*Blimbing*):** El algoritmo comienza con una solución inicial y luego le aplica pequeñas mejoras sucesivas hasta llegar a la mejor solución (Se suele aplicar para encontrar un máximo) [[Hill Climbing](#)]
- **Búsqueda por máxima pendiente:** Similar al anterior pero realiza evaluaciones previas de varios nodos para evitar los máximos locales
- **Primero mejor o Best First:** Es una combinación de profundidad y a lo ancho. Va generando los sucesores del estado actual y los deja en una lista de estados sin revisar. Evalúa al más prometedor si es final termina la búsqueda sino pasa a ser estado actual.
- **Método de búsqueda A*:** Busca el camino en un grafo desde un vértice inicial a un vértice final. Analiza los distintos recorridos quedándose con el de menor costo de $f(n)$. Se guía por una función: $F(n) = g(n) + h(n)$.
Siendo $g(n)$ = costo del camino hasta n

Siendo $h(n)$ = heurística del nodo, estimación del costo de un camino óptimo desde n hasta un estado final.

Fin primer parcial.